

Traduction des langages

TD4 placement mémoire

1 AST Typage

```
module AstType =
struct

  (* Opérateurs unaires de Rat – résolution de la surcharge *)
  type unaire = Numerateur | Denominateur

  (* Opérateurs binaires existants dans Rat – résolution de la surcharge *)
  type binaire = Fraction | PlusInt | PlusRat | MultInt | MultRat | EquInt | EquBool | Inf

  (* Expressions existantes dans Rat *)
  (* = expression de AstType *)
  type expression =
    | AppelFonction of Tds.info_ast * expression list
    | Ident of Tds.info_ast
    | Booleen of bool
    | Entier of int
    | Unaire of unaire * expression
    | Binaire of binaire * expression * expression

  (* instructions existantes Rat *)
  (* = instruction de AstType + informations associées aux identificateurs, mises à jour *)
  (* + résolution de la surcharge de l’affichage *)
  type bloc = instruction list
  and instruction =
    | Declaration of Tds.info_ast * expression
    | Affectation of Tds.info_ast * expression
    | AffichageInt of expression
    | AffichageRat of expression
    | AffichageBool of expression
    | Conditionnelle of expression * bloc * bloc
    | TantQue of expression * bloc
    | Retour of expression * Tds.info_ast
    | Empty (* les nœuds ayant disparus: Const *)

  (* informations associées à l’identificateur (dont son nom), liste des paramètres, corps *)
  type fonction = Fonction of Tds.info_ast * Tds.info_ast list * bloc

  (* Structure d’un programme dans notre langage *)
  type programme = Programme of fonction list * bloc

end
```

2 Signatures de la passe Placement mémoire

```
(* AstType.instruction -> int -> string -> AstPlacement.instruction * int *)
(* instruction, depl, reg -> instruction * taille *)
let rec analyse_placement_instruction i depl reg = ...
(* AstType.bloc -> int -> string -> AstPlacement.bloc *)
  and analyse_placement_bloc li depml reg = ...

(* AstType.fonction -> AstPlacement.fonction *)
let analyse_placement_fonction (AstType.Fonction(info,lp,li)) = ...

(* AstType.programme -> AstPlacement.programme *)
let analyser (AstType.Programme (fonctions, prog)) = ...
```

3 Squelette de analyse_type_instruction

```
(* AstType.instruction -> int -> string -> AstPlacement.instruction * int *)
(* instruction, depl, reg -> instruction * taille *)
let rec analyse_placement_instruction i depl reg =
  match i with
  | AstType.Declaration (info,e) ->
    begin

    end
  | AstType.Conditionnelle (c,t,e) ->

  | AstType.TantQue (c,b) ->

  | AstType.Retour (e, ia) ->

  | AstType.Affectation (ia,e) ->

  | AstType.AffichageInt e ->

  | AstType.AffichageRat e ->
```

```
| AstType.AffichageBool e ->
```

```
| AstType.Empty ->
```

and

```
(* analyse_placement_bloc : AstType.bloc -> int -> string -> AstPlacement.bloc *)  
analyse_placement_bloc li repl reg =
```

4 Squelette de analyser et analyse_placement_fonction

```
(* AstType.programme -> AstPlacement.programme *)  
let analyser (AstType.Programme (fonctions,prog)) =
```

```
let analyse_placement_fonction (AstType.Fonction(info,lp,li)) =
```